ECEN 749 Lab 5 - Report

Yashas Bharadwaj
UIN: 635009332; Section: 601

Introduction

This lab focuses on building kernel modules and then loading it onto linux that will be deployed on the Zybo-Z7 board via SD card booting. The lab involves creating two kernel modules; hello and multiply. The latter will be used to access the multiplication peripheral that was previously created. The lab then involves combining the generated .ko files along with the 3 files for booting into the SD card. All the files were used for the boot process of linux and to load and run the kernel modules on the Zybo board.

Procedure

1. To mount the SD card onto the FPGA, we use picocom console. And this is tested by creating a directory in /mnt. Unmount /mnt is used to unmount the SD card's FAT partition to prevent corruption.

2. The next step is to create the kernel modules on the SD card through the linux PC, this is done by taking reference of the previous lab, by creating a project template after sourcing the petalinux settings. Then the petalinux was installed in the project directory created. But in this lab, petalinux-build was not performed. Create a module using the petalinux-create command.

3. Two such modules need to be created; hello.c, and multiply.c. The latter file had to be modified to map, display the physical and virtual address and to write and read from the registers. In the multiply module, the xparameters.h and xparameters_ps.h needs to be added to the multiply.bb file.

4. After this, petalinux-build is executed. This generates the necessary .ko files which need to be copied to the SD card. From the picocom console, the SD card was mounted onto the Zybo board. Insmod /mnt/<filename>.ko was used to load the module into the linux kernel.

5. dmesg | tail is used to display the C file outputs on the console.

Results

The picocom console on the terminal displayed the linux boot by prompting a user login and password. The working of the kernel was validated by sudo su command to get access to root user and then after using the insmod and dmesg | tail commands, the required outputs were visible. The modifications made to the multiply.c file include using the ioremap () and iowrite32() and printk statement to display the physical and virtual addresses.

Conclusion

This lab provided valuable experience in understanding and building linux kernel modules using petalinux tools. This also helped me understand how to develop kernel modules using C language and run it on the Zybo board. The picocom console was used to see the output of the kernel module and gives a good insight into understanding how kernel modules work in a petalinux system.

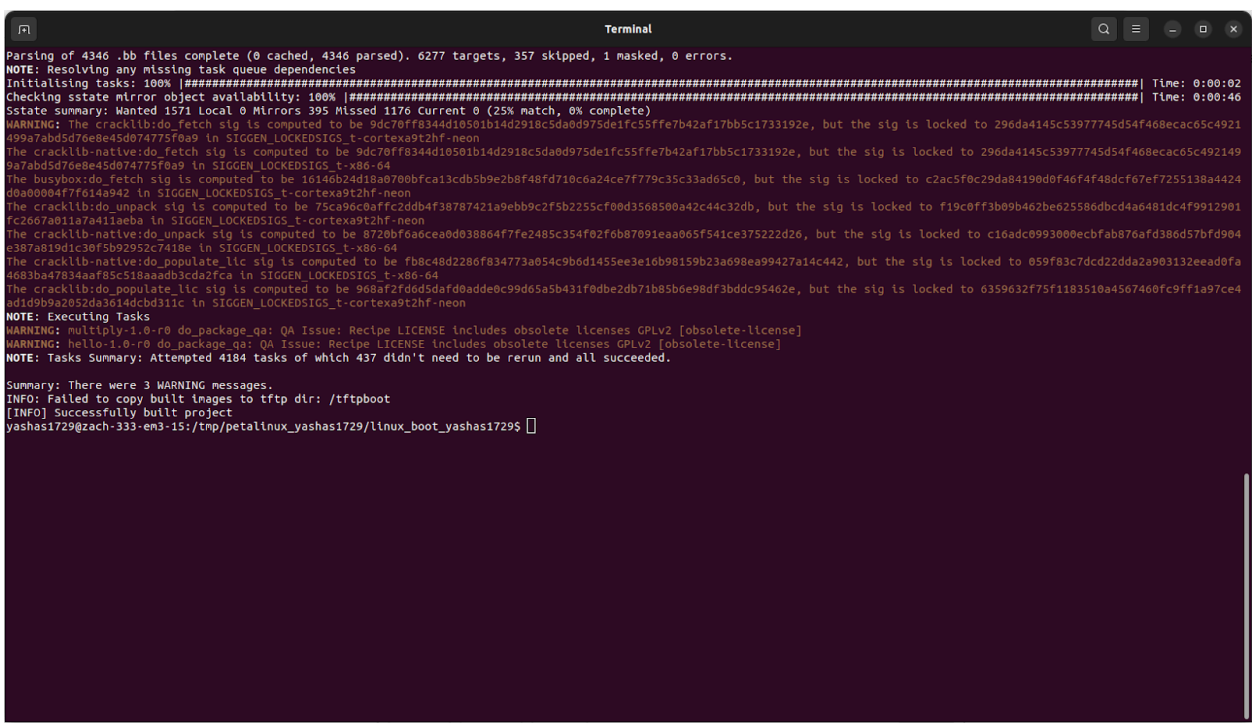
Questions

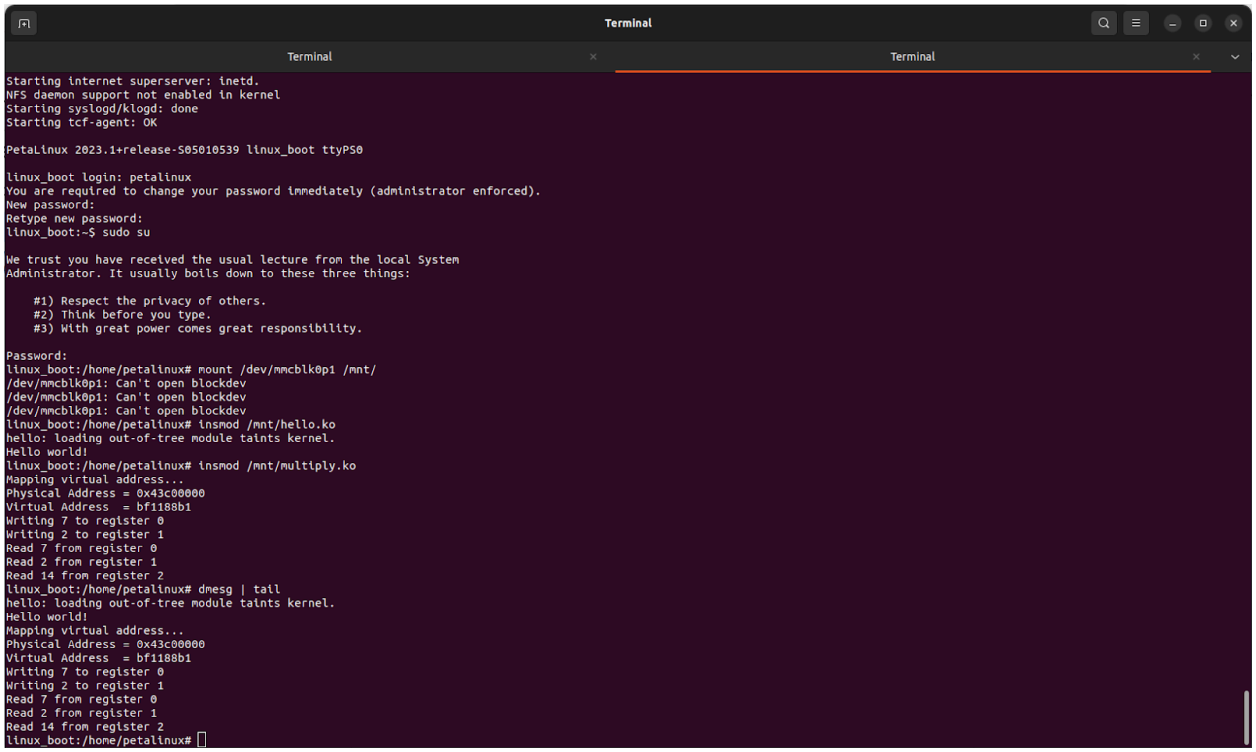
a) "/media/yashas1729/ext4" is where the SD card is present on the linux machine.

b) If the hello.c file was changed to x.c then we need to just change the name of the object file to obj-m = a.o
Even if the same kernel directory is specified as lab 4, it will be okay as KERNEL_SRC points to the same kernel version in both the labs.

Appendix

Picocom console:





```
Starting internet superserver: inetd.
NFS daemon support not enabled in kernel
Starting syslogd/klogd: done
Starting tcf-agent: OK

PetaLinux 2023.1+release-S05010539 linux_boot ttyPS0

linux_boot login: petalinux
You are required to change your password immediately (administrator enforced).
New password:
Retype new password:
linux_boot:~$ sudo su

We trust you have received the usual lecture from the local System
Administrator. It usually boils down to these three things:

    #1) Respect the privacy of others.
    #2) Think before you type.
    #3) With great power comes great responsibility.

Password:
linux_boot:/home/petalinux# mount /dev/mmcblk0p1 /mnt/
/dev/mmcblk0p1: Can't open blockdev
/dev/mmcblk0p1: Can't open blockdev
/dev/mmcblk0p1: Can't open blockdev
linux_boot:/home/petalinux# insmod /mnt/hello.ko
hello: loading out-of-tree module taints kernel.
Hello world!
linux_boot:/home/petalinux# insmod /mnt/multiply.ko
Mapping virtual address...
Physical Address = 0x43c00000
Virtual Address  = bf1188b1
Writing 7 to register 0
Writing 2 to register 1
Read 7 from register 0
Read 2 from register 1
Read 14 from register 2
linux_boot:/home/petalinux# dmesg | tail
hello: loading out-of-tree module taints kernel.
Hello world!
Mapping virtual address...
Physical Address = 0x43c00000
Virtual Address  = bf1188b1
Writing 7 to register 0
Writing 2 to register 1
Read 7 from register 0
Read 2 from register 1
Read 14 from register 2
linux_boot:/home/petalinux#
```

“Multiply.c” code:

```
#include <linux/module.h>   // Needed by all modules
#include <linux/kernel.h>   // Needed for KERN_* and printk
#include <linux/init.h>     // Needed for __init and __exit macros
#include <asm/io.h>         // Needed for IO reads and writes
#include "xparameters.h"    // AXI physical addresses
#include <linux/ioport.h>   // IO memory allocation

// Physical address of multiplier
#define PHY_ADDR    XPAR_MULTIPLY_0_S00_AXI_BASEADDR

// Size of physical address range
#define MEMSIZE     (XPAR_MULTIPLY_0_S00_AXI_HIGHADDR - \
                     XPAR_MULTIPLY_0_S00_AXI_BASEADDR + 1)

static void *virt_addr;  // virtual address

static int __init my_init(void)
{
    printk(KERN_INFO "Mapping virtual address...\n");

    // Map physical → virtual
    virt_addr = ioremap(PHY_ADDR, MEMSIZE);
```

```
    if (!virt_addr) {
        printk(KERN_ALERT "ERROR: ioremap failed!\n");
        return -ENOMEM;
    }

    printk(KERN_INFO "Physical Address = 0x%1x\n",(unsigned long)PHY_ADDR);
    printk(KERN_INFO "Virtual Address  = %p\n", virt_addr);

    // Write 7 to register 0
    printk(KERN_INFO "Writing 7 to register 0\n");
    iowrite32(7, virt_addr + 0);

    // Write 2 to register 1
    printk(KERN_INFO "Writing 2 to register 1\n");
    iowrite32(2, virt_addr + 4);

    // Read registers
    printk(KERN_INFO "Read %d from register 0\n", ioread32(virt_addr + 0));
    printk(KERN_INFO "Read %d from register 1\n", ioread32(virt_addr + 4));
    printk(KERN_INFO "Read %d from register 2\n", ioread32(virt_addr + 8));

    return 0;
}

static void __exit my_exit(void)
{
    printk(KERN_ALERT "Unmapping virtual address...\n");
    iounmap(virt_addr);
}

MODULE_LICENSE("GPL");
MODULE_AUTHOR("ECEN749 Student");
MODULE_DESCRIPTION("Simple multiplier module");

module_init(my_init);
module_exit(my_exit);
```

Sample code given:

```
/* hello.c – Simple multiplier kernel module
 *
```

```
 * Demonstrates module initialization, physical memory mapping, IO operations, and cleanup.
 *
 * Adapted from various example modules.
 */

#include <linux/module.h>   /* Needed by all modules */
#include <linux/kernel.h>   /* Needed for printk */
#include <linux/init.h>     /* Needed for __init and __exit macros */
#include <asm/io.h>         /* Needed for IO reads and writes */
#include "xparameters.h"    /* Needed for physical address of multiplier */

/* From xparameters.h */
#define PHY_ADDR_XPAR_MULTIPLY_0_S00_AXI_BASEADDR 0x40000000  /* Physical
address of multiplier */

/* Size of physical address range for multiplier */
#define MEMSIZE (XPAR_MULTIPLY_0_S00_AXI_HIGHADDR -
XPAR_MULTIPLY_0_S00_AXI_BASEADDR + 1)

void *virt_addr;  /* Virtual address pointing to multiplier */

/* This function runs upon module load. Set up data structures and
 * reserve resources used by the module here.
 */
static int __init myinit(void)
{
    printk(KERN_INFO "Mapping virtual address...\n");

    /* Map the virtual address to the multiplier's physical address */
    virt_addr = ioremap(PHY_ADDR_XPAR_MULTIPLY_0_S00_AXI_BASEADDR, MEMSIZE);
    if (!virt_addr) {
        printk(KERN_ALERT "ioremap failed!\n");
        return -ENOMEM;
    }

    /* Write 7 to register 0 */
    printk(KERN_INFO "Writing a 7 to register 0\n");
    iowrite32(7, virt_addr + 0);  /* Base address + offset for register 0 */

    /* Write 2 to register 1 */
    printk(KERN_INFO "Writing a 2 to register 1\n");
    iowrite32(2, virt_addr + 4);  /* Offset for register 1 */

    /* Read from register 0 */
```

```
    printk("Read %d from register 0\n", ioread32(virt_addr + 0));

    /* Read from register 1 */
    printk("Read %d from register 1\n", ioread32(virt_addr + 4));

    /* Read from register 2 */
    printk("Read %d from register 2\n", ioread32(virt_addr + 8));

    return 0;  /* Return 0 means initialization was successful */
}

/* This function is run just prior to the module's removal from the system.
 * Release ALL resources used by your module here.
 */
static void __exit myexit(void)
{
    printk(KERN_ALERT "Unmapping virtual address space...\n");
    iounmap(virt_addr);  /* Unmap the virtual address */
}

/* Module metadata */
MODULE_LICENSE("GPL");
MODULE_AUTHOR("ECEN449 Student (and others)");
MODULE_DESCRIPTION("Simple multiplier module");

/* Define which functions to use for initialization and cleanup */
module_init(myinit);
module_exit(myexit);
```